

Exp No: 4**Implementation of Symbol Table****Aim:**

To implement a symbol table using C programming that stores the **symbol, address, data type, size, and offset** of variables declared in declaration statements.

Algorithm:

1. Start the program.
2. Define a structure to store symbol, address, data type, size, and offset.
3. Read the number of declaration statements.
4. Read each declaration.
5. Extract the data type and variable names.
6. Determine the size of each data type.
7. Assign an offset to each variable starting from 0.
8. Calculate the address using the base address and offset.
9. Store all information in the symbol table.
10. Display the complete symbol table.
11. Stop the program.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
struct Symbol
{
    char name[20];
    unsigned long address;
    char type[20];
    int size;
    int offset;
};
```

```
struct Symbol table[50];
int count = 0;
int offset = 0;
```

```
int getSize(char type[])
{
    if (strcmp(type, "int") == 0)
        return 4;

    if (strcmp(type, "float") == 0)
        return 4;

    if (strcmp(type, "char") == 0)
        return 1;
```

```

    if (strcmp(type, "double") == 0)
        return 8;

    return 0;
}

void processDeclaration(char line[])
{
    char type[20];
    char *token;
    char *rest;

    token = strtok(line, " \t");

    if (token == NULL)
        return;

    strcpy(type, token);

    rest = strtok(NULL, "");

    if (rest == NULL)
        return;

    token = strtok(rest, ",");

    while (token != NULL)
    {
        char name[20];
        int i = 0, j = 0;
        int size;

        while (token[i] != '\0')
        {
            if (!isspace((unsigned char)token[i]))
                name[j++] = token[i];

            i++;
        }

        name[j] = '\0';

        if (j > 0 && name[j - 1] == ';')
            name[j - 1] = '\0';

        if (strlen(name) > 0)
        {
            size = getSize(type);

            strcpy(table[count].name, name);

```

```

        strcpy(table[count].type, type);

        table[count].size = size;
        table[count].offset = offset;
        table[count].address = 1000 + offset;

        offset = offset + size;
        count++;
    }

    token = strtok(NULL, ",");
}
}

int main()
{
    int n, i;
    char line[100];

    printf("Enter the number of declarations: ");
    scanf("%d", &n);

    getchar();

    printf("\nEnter the declarations:\n");

    for (i = 0; i < n; i++)
    {
        fgets(line, sizeof(line), stdin);
        processDeclaration(line);
    }

    printf("\n----- SYMBOL TABLE ----- \n");

    printf("%-12s %-12s %-12s %-8s %-8s\n",
           "Symbol", "Address", "Type", "Size", "Offset");

    printf("----- \n");

    for (i = 0; i < count; i++)
    {
        printf("%-12s %-12lu %-12s %-8d %-8d\n",
               table[i].name,
               table[i].address,
               table[i].type,
               table[i].size,
               table[i].offset);
    }

    return 0;
}

```

```
}
```

Procedure to Execute

Step 1: Save the Program

Open **Notepad**, paste the program and save it as:
symbol.c

Select **Save as type** → **All Files**.

Step 2: Open CMD

cd P:\pcd_lab\exp_4

Step 3: Compile

gcc symbol.c -o symbol.exe

Step 4: Run

symbol.exe

Enter:

2

int a,b

float c,d

Output:

```
Command Prompt

P:\pcd_lab\exp_4>gcc symbol.c -o symbol.exe

P:\pcd_lab\exp_4>symbol.exe
Enter the number of declarations: 2

Enter the declarations:
int a,b
float c,d

----- SYMBOL TABLE -----
Symbol      Address      Type      Size      Offset
-----
a           1000         int        4          0
b           1004         int        4          4
c           1008         float      4          8
d           1012         float      4         12

P:\pcd_lab\exp_4>|
```

Result:

Thus, the **Symbol Table** was successfully implemented in C, and the variables declared were stored with their **symbol, address, data type, size, and offset**.